Edge Intelligence

Assessment - Lab 8

M Rahul – 25MML0011

1. Webcam Video Capture

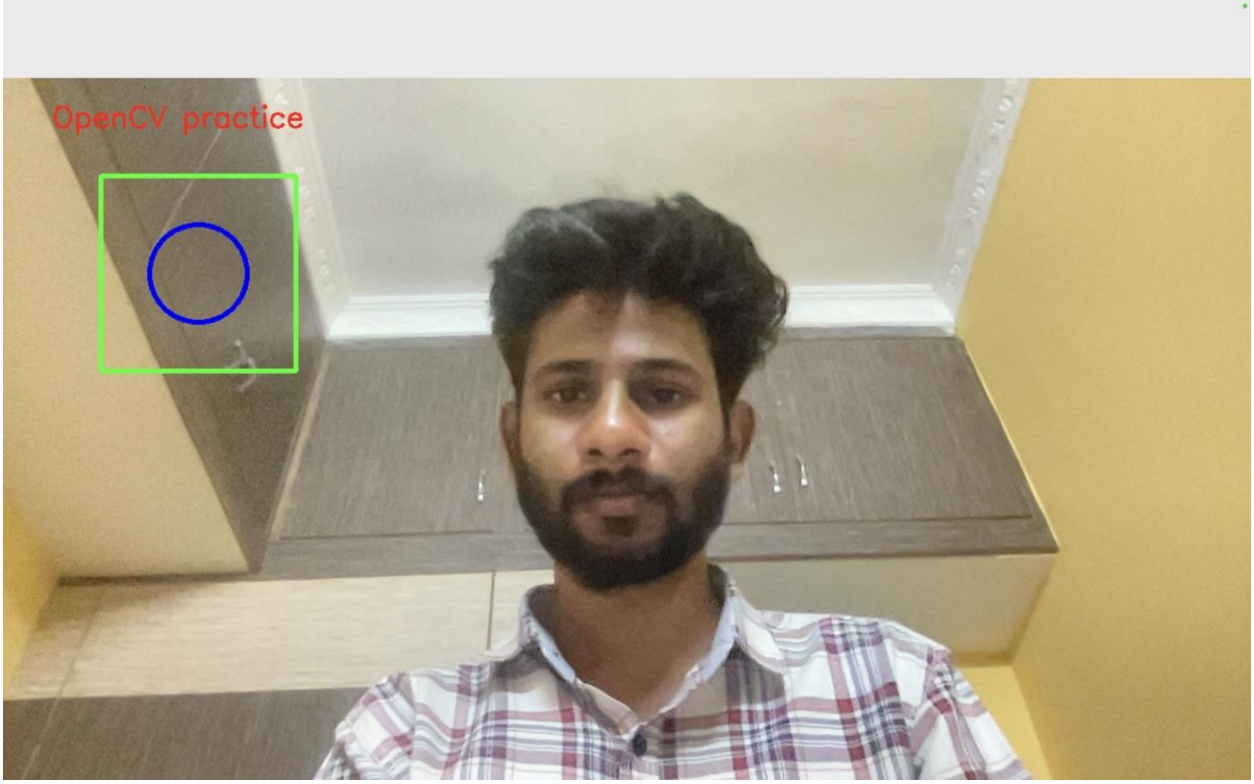
[3]:
```
import cv2
```

[6]:
```
import cv2

cap = cv2.VideoCapture(0, cv2.CAP_AVFOUNDATION)

print("Camera opened:", cap.isOpened())

while True:
    ret, frame = cap.read()

    if not ret:
        print("Failed to grab frame")
        break

    cv2.imshow("Mac Webcam", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

```
Camera opened: True
```

2. Drawing Shapes (Rectangle & Circle)

[8]:
```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    cv2.rectangle(frame, (100,100),(300,300),(0,255,0),3)
    cv2.circle(frame,(200,200),50,(255,0,0),3)

    cv2.putText(frame,"OpenCV practice",(50,50),cv2.FONT_HERSHEY_SIMPLEX,1,(0,0,255),2)

    cv2.imshow("Shapes",frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

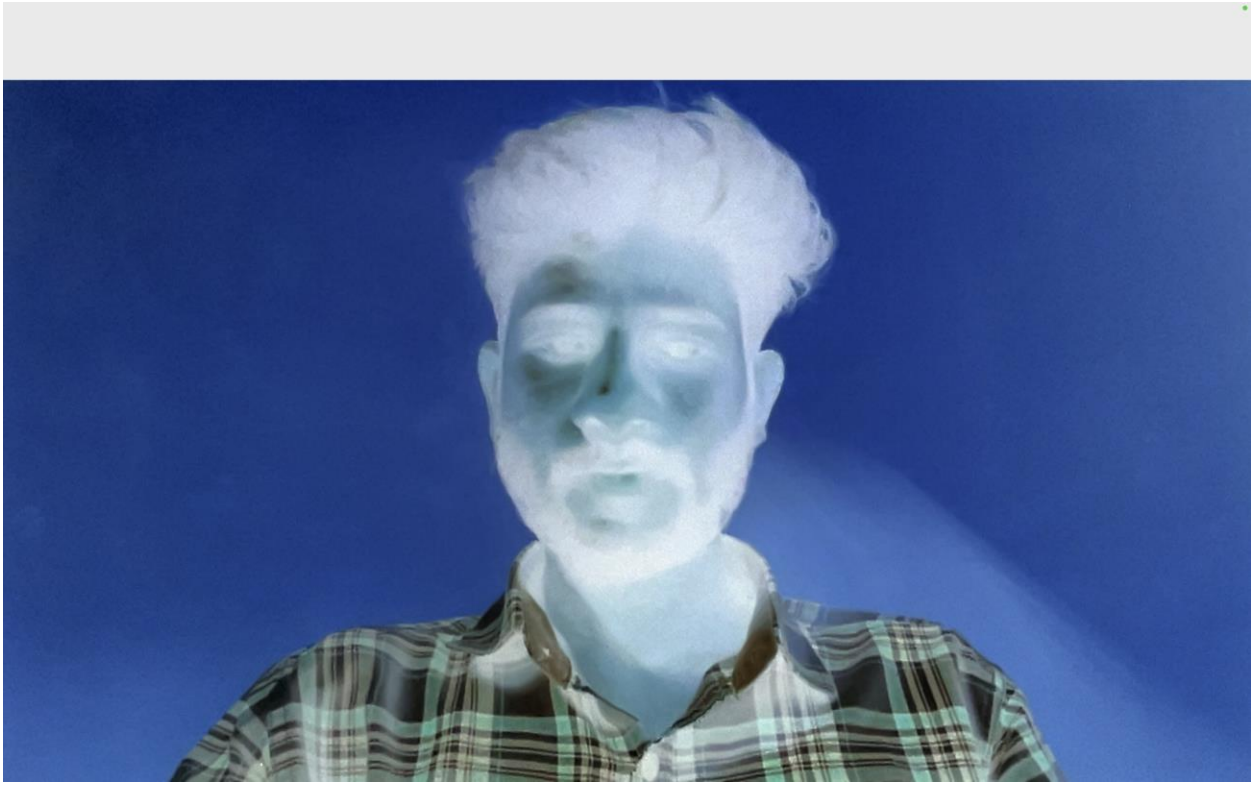


3. Canny Edge Detection

[19]:
```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    edges = cv2.Canny(frame,100,200)

    cv2.imshow("Edges",edges)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

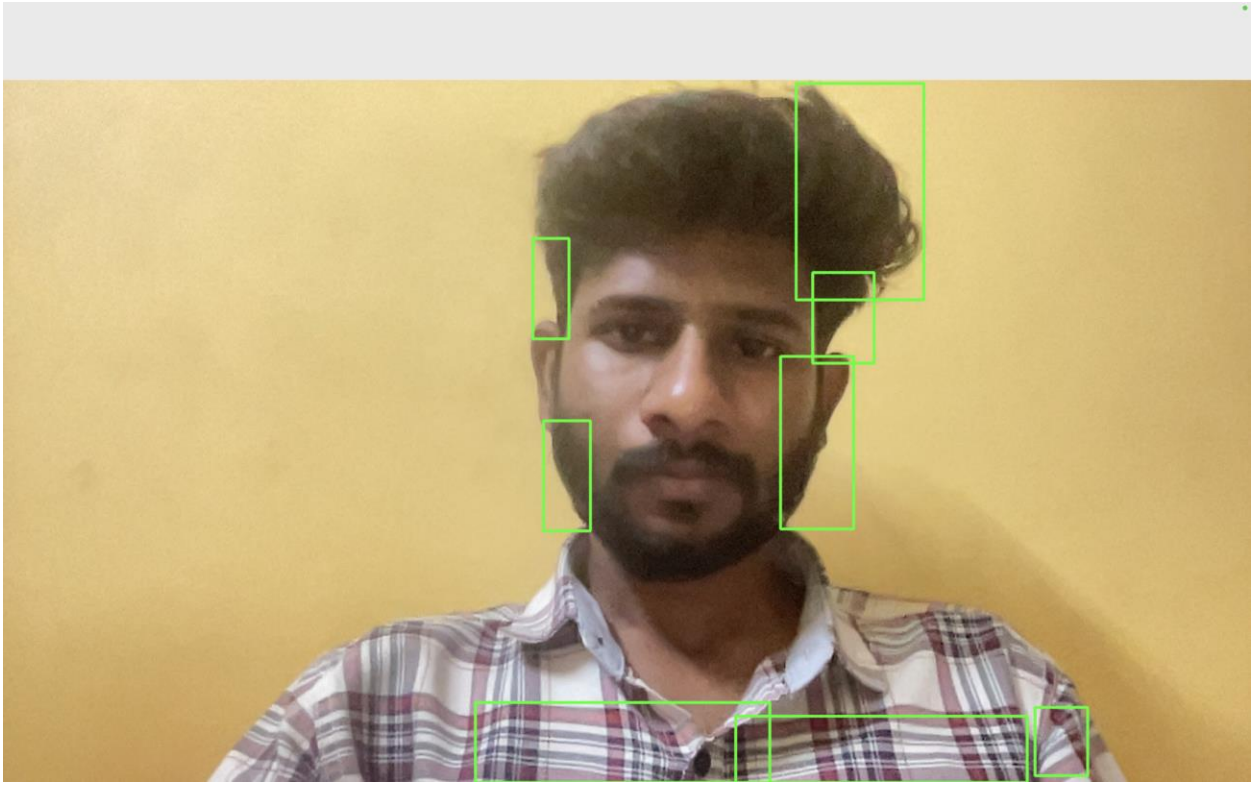
4. Negative Image Transformation

```
[10]: import cv2

cap = cv2.VideoCapture(0, cv2.CAP_AVFOUNDATION)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    # Negative transformation
    negative = 255 - frame

    cv2.imshow("Original", frame)
    cv2.imshow("Negative Image", negative)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



5. Motion Detection

```
[24]: import cv2

cap = cv2.VideoCapture(0, cv2.CAP_AVFOUNDATION)

ret, frame1 = cap.read()
ret, frame2 = cap.read()

while cap.isOpened():

    # Frame difference
    diff = cv2.absdiff(frame1, frame2)

    # Convert to grayscale
    gray = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)

    # Blur to remove noise
    blur = cv2.GaussianBlur(gray, (5,5), 0)

    # Threshold
    _, thresh = cv2.threshold(blur, 20, 255, cv2.THRESH_BINARY)

    # Dilate to fill gaps
    dilated = cv2.dilate(thresh, None, iterations=3)

    # Find contours (motion areas)
    contours, _ = cv2.findContours(dilated, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

    for contour in contours:
        if cv2.contourArea(contour) < 1000:
            continue

        x,y,w,h = cv2.boundingRect(contour)

        cv2.rectangle(frame1,(x,y),(x+w,y+h),(0,255,0),2)

    cv2.imshow("Motion Detection", frame1)

    frame1 = frame2
    ret, frame2 = cap.read()

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



```python
import cv2
from google.colab.patches import cv2_imshow

cap = cv2.VideoCapture("sample_video.mp4")

while cap.isOpened():
    ret, frame = cap.read()

    if not ret:
        break

    cv2_imshow(frame)

    if cv2.waitKey(25) & 0xFF == ord('q'):
        break

cap.release()
```

[2]
0s

```python
import cv2

cap = cv2.VideoCapture("sample.mp4")

total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
fps = cap.get(cv2.CAP_PROP_FPS)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)

print("Total frames:", total_frames)
print("FPS:", fps)
print("Width:", width)
print("Height:", height)

cap.release()
```

```
Total frames: 2398
FPS: 59.94005994005994
Width: 1280.0
Height: 720.0
```

```python
import cv2
from google.colab.patches import cv2_imshow

cap = cv2.VideoCapture("sample.mp4")

while True:
    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    cv2_imshow(gray)

cap.release()
```

```python
import cv2

cap = cv2.VideoCapture("sample.mp4")

ret, frame = cap.read()

if ret:

    # Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    print("Original Image Matrix (BGR):")
    print(frame)

    print("\nGrayscale Image Matrix:")
    print(gray)

cap.release()
```

```
Original Image Matrix (BGR):
[[[ 54  67  61]
  [ 54  67  61]
  [ 54  67  61]
  ...
  [ 19  13   8]
  [ 19  13   8]
  [ 19  13   8]]

 [[ 54  67  61]
  [ 54  67  61]
  [ 54  67  61]
  ...
  [ 19  13   8]
  [ 19  13   8]
  [ 19  13   8]]
```

```
  [[127 122 123]
   [126 121 122]
   [125 120 121]
   ...
   [ 38   33   34]
   [ 38   33   34]
   [ 38   33   34]]

  [[128 123 124]
   [125 120 121]
   [119 114 115]
   ...
   [ 40   35   36]
   [ 40   35   36]
   [ 40   35   36]]

  [[116 111 112]
   [112 107 108]
   [104   99 100]
   ...
   [ 40   35   36]
   [ 40   35   36]
   [ 40   35   36]]]

Grayscale Image Matrix:
[[ 64   64   64 ...   12   12   12]
 [ 64   64   64 ...   12   12   12]
 [ 61   61   61 ...   12   12   12]
 ...
 [123 122 121 ...   34   34   34]
 [124 121 115 ...   36   36   36]
```

Steps :-

1. **Import OpenCV Library**
   import cv2 – Loads the OpenCV library used for image and video processing.
2. **Open Video File**
   cv2.VideoCapture("sample_video.mp4") – Opens the video file and prepares it for frame-by-frame processing.
3. **Read Video Frames**
   cap.read() – Reads one frame from the video and returns the frame matrix and status.
4. **Display Video Frame**
   cv2.imshow() – Displays the video frame in a window for visualization.
5. **Control Video Playback**
   cv2.waitKey() – Waits for a key press and controls the speed of frame display.
6. **Close Video and Windows**
   cap.release() and cv2.destroyAllWindows() – Releases the video object and closes all display windows.
7. **Get Video Properties**
   cap.get() – Retrieves video metadata like total frames, FPS, width, and height.
8. **Convert Frame to Grayscale**
   cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) – Converts a color frame to grayscale to simplify processing.
9. **Print Pixel Matrix**
   print(frame) / print(gray) – Displays the pixel intensity values of the image matrix.
10. **Edge Detection**
    cv2.Canny() – Detects object edges in the frame using gradient-based edge detection.
11. **Negative Image Transformation**
    255 - frame – Creates the negative of the image by inverting pixel intensities.
12. **Motion Detection**
    cv2.absdiff(frame1, frame2) – Calculates the difference between two frames to detect movement.
13. **Thresholding**
    cv2.threshold() – Converts grayscale images into binary images for easier object detection.

14. **Contour Detection**

cv2.findContours() – Detects boundaries of moving objects or shapes in the

Mathematical Equation :-

<u>Image Representation :-</u>

An image is a matrix of pixels.

$$F(x, y) = [B, G, R]$$

Where

- x, y = pixel coordinates
- B, G, R = color intensity values (0–255)

Image size:

$$Image = Height \times Width \times Channels$$

<u>Negative Image Transformation :-</u>

$$S = L - 1 - R$$

For 8-bit images:

$$S = 255 - R$$

Where

- R = original pixel
- S = negative pixel

<u>Grayscale Conversion :-</u>

$$Gray = 0.299R + 0.587G + 0.114B$$

This converts a **3-channel color image → 1-channel grayscale image**.

<u>Image Gradient :-</u>

$$G = \sqrt{G_x^2 + G_y^2}$$

Where

- G_x = horizontal gradient
- G_y = vertical gradient

Used in **Canny edge detection**.

<u>Frame Difference (Motion Detection) :-</u>

$$D(x, y) = |F_t(x, y) - F_{t-1}(x, y)|$$

Where

- F_t = current frame
- F_{t-1} = previous frame

If

$$D(x, y) > T$$

motion is detected.